॥श्री गणेशाय नमः॥

Classification

Classification is a type of supervised learning where the goal is to assign a label/class to an input based on its features.
E.g. spam detection
In classification, the input is a set of features, and the output is a distinct category. The model learns a mapping between the features and the class labels based on the labeled dataset.
It is a task of predicting categorical outputs.

$D = \{(x_i, y_i)\}_{i=1}^{n}$ $\quad x_i \rightarrow$ features, $y_i \rightarrow$ class label associated with x_i

objective: learn $f(x)$ that predicts y for an unseen x.

Classification can be:
- Binary classification — 2 classes
- Multiclass classification — >2 classes
  - ↳ Multilabel classification — Each instance can belong to multiple classes simultaneously.

Classification (datasets):
- Balanced: same order/# of examples for every class
- Imbalanced: minority/majority classes exist

Input interpretation in classification

Input features:
- Numerical features → Represent quantity/measurements; can take a wide range of values. E.g. temperature.
- Categorical features → represent data that can be divided into categories/groups. E.g. sex
- Textual or image features → require special handling, like word embeddings.

Handling categorical features

Categorical features cannot be directly used

Therefore categorical features must be converted to numerical features.

categorical data:
- ordinal (data can be ordered)
- Nominal (no order in the categories)

1. Label encoding: Each category is assigned a numerical (integer) value.

works on ordinal data.

not confident	slightly confident	very confident
1	2	3

For nominal data, label encoding can create misleading data.

2. One-hot encoding: It creates binary features for each category (1=presence, 0=absence of category).

colors = {red, green, blue}

R	G	B	
1	0	0	→ Red
0	1	0	→ green
0	0	1	→ Blue

Pros:
- eliminates issues of ordinal relationships.

Cons:
- increases the dimensionality of input and model.

Handling numerical features

① Feature scaling
- min-max scaling: $x_{scaled} = \frac{x - \min(x)}{\max(x) - \min(x)}$
- standardization (z-score normalization): $x' = \frac{x - \mu}{\sigma}$

② Handling outliers
- clipping: restrict values above and below a threshold
- log transformation: it reduces the impact of extreme values by compressing the scale of very large #s.

Dealing with missing data

Removal:
- Remove rows with an attribute missing.
- Remove columns/features with a lot of data values missing.
- Can lead to loss of information.

Imputation:
- Filling the missing values with estimates (mean, mode, median).

Output interpretation in classification

In classification, the model's output can take various forms depending on the algo used.
- Class probabilities: A prob distribution over possible class labels. In logistic regression, neural nets, etc.
- Labels: A direct assignment to one of the possible outcomes. In decision trees, KNN, etc.

Thresholding

In classification, the output is often a probability value and a decision needs to be made about whether to classify the instance as 1 or 0.
↳ can be achieved using ROC curve.

Softmax function (multi-class)

The output of an ML algo can be a vector of unnormalized scores (logits) for each class. These must be converted to probabilities.
The softmax function transforms raw scores (logits) into probabilities that sum upto 1, making it suitable for multi-class classification.

$$\sigma(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_i}}$$

$z_i \rightarrow$ raw score for i^{th} class
$K \rightarrow$ # of classes
$\sigma(z)_i \rightarrow$ prob. of ith class

e.g.

	cat	dog	bird
z	2	1	0.1

$$P(cat) = \sigma(cat) = \frac{e^2}{e^2+e+e^{0.1}} = 0.66$$

$$\sigma(dog) = \frac{e}{e^2+e+e^{0.1}} = 0.24$$

$$\sigma(bird) = \frac{e^{0.1}}{e^2+e+e^{0.1}} = 0.1$$

Sigmoid function (binary)

For +ve class, we get a single logit.

$$\sigma(+ve) = \frac{1}{1+e^{-z}}$$

E.g. for multi-label classificatn with following logits, what classes does the input label correctly if threshold prob = 60% ?
cat = 1.5, dog = -0.5, bird = 2.0. use sigmoid.

→ $\sigma(cat) = \frac{1}{1+e^{-1.5}} = 0.8175$

$\sigma(dog) = \frac{1}{1+e^{0.5}} = 0.3775$

$\sigma(bird) = \frac{1}{1+e^{-2}} = 0.8807$

cat ✓
bird ✓

Error correcting output codes

For 3 class problem :

output code would be

	f_0	f_1	f_2	f_3	f_4	f_5	f_6	f_7
A	0	0	0	0	1	1	1	1
B	0	0	1	1	0	0	1	1
C	0	1	0	1	0	1	0	1

} hamming distance = 4

If a bit goes wrong,
11010101 ← easy to judge that it's from class C.

so the 1st output code is much better.

output code could also be :

A	1	0	0
B	0	1	0
C	0	0	1

} hamming distance = 2

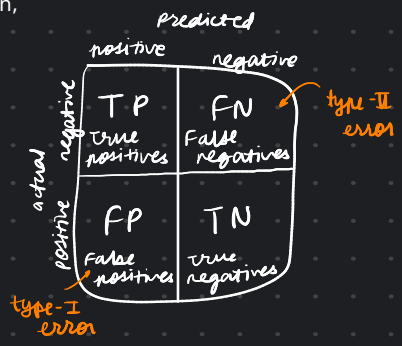
If a bit got wrong: 110
we can't decide which class it is in.

Error metrics for classification

Confusion matrix

- A confusion matrix displays the # of correct and incorrect predictions for each class broken down by the actual and predicted classes.

For binary classification,



N total observations

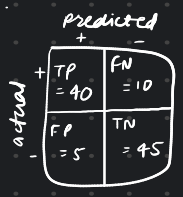
Say we want to predict if a patient has a tumor (+ve) or not (-ve).
we would prefer a lesser # of false negatives. This preference is based on the problem statement.

Accuracy

- The ratio of correctly predicted observations (+ve and -ve) to the total # of observations.

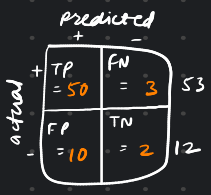
$$\text{Accuracy} = \frac{TP+TN}{TP+TN+FP+FN} = \frac{TP+TN}{\text{total}}$$

e.g.



$$\text{Accuracy} = \frac{40+45}{100} = 0.85$$

- Accuracy can be misleading in imbalanced classes (one of them is much more frequent than the other).



Accuracy = 0.80
Here actual -ves are 12.
only 2 of those are predicted rightly.
Predicted +ve values = 60
predicted -ve values = 5

The model is leisaed towards the +ve class. It doesn't predict -ve much.
Basically it doesn't learn patterns, just predicts +ve in most cases.
And there's imbalanced data a lot, like most people won't have a tumor.

Precision

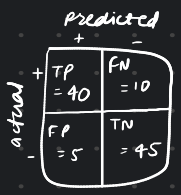
- Of all the instances predicted as positive, how many were actually positive?
- Ratio of correctly predicted positives to all all the positive predictions.

$$\text{Precision} = \frac{TP}{TP+FP} = \frac{TP}{\hat{P}}$$

Recall/sensitivity

- Of all the +ve instances, how many were identified correctly?

$$\text{Recall} = \frac{TP}{TP+FN} = \frac{TP}{P}$$

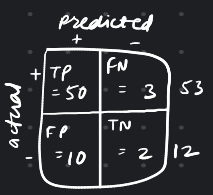


$$\text{Accuracy} = \frac{40+45}{100} = 0.85$$

$$\text{Precision} = \frac{40}{40+5} \approx 0.8888$$

$$\text{Recall} = \frac{40}{40+10} = 0.80$$

In case of class imbalance,



Accuracy = 0.80

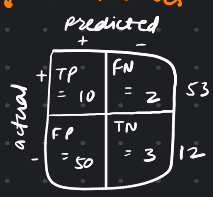
$$\text{Precision} = \frac{50}{50+10} \approx 0.83$$

$$\text{Recall} = \frac{50}{50+3} = \frac{50}{53} \approx 0.9433$$

$$\text{TNR} = \frac{2}{10+2} = \frac{1}{6} = 0.167$$ Bad.

High cause imbalance is on the +ve side.

Real data will have more -ves.



$$\text{accuracy} = \frac{60}{65}$$

$$\text{Precision} = \frac{10}{10+50} = \frac{10}{60}$$ low precision.

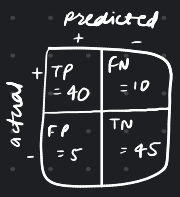
$$\text{Recall} = \frac{10}{10+2} = \frac{10}{12}$$

we'll calculate precision and recall w.r.t. both precision and recall.

F1 score

- It is the harmonic mean of the precision and the recall.
- A higher f1-score indicates better performance w.r.t. both precision and recall.

$$\text{F1-score} = 2 \cdot \frac{\text{precision} \times \text{recall}}{\text{Precision} + \text{recall}}$$



$$\text{Accuracy} = \frac{40+45}{100} = 0.85$$

$$\text{Precision} = \frac{40}{40+5} \approx 0.8888$$

$$\text{Recall} = \frac{40}{40+10} = 0.80$$

$$\text{F1 score} = 2 \cdot \frac{\frac{40}{45} \times \frac{40}{50}}{\frac{40}{45} + \frac{40}{50}} = \frac{2 \cdot \frac{8}{9} \cdot \frac{8}{10}}{8\left(\frac{1}{9}+\frac{1}{10}\right)} = \frac{16}{90} \times \frac{90}{19} = \frac{16}{19} = 0.8421$$

True negative rate (specificity)

- Of all the -ve instances, how many were correctly identified?

$$\text{TNR} = \frac{TN}{TN+FP} = \frac{TN}{N} \leftarrow \text{all -ves}$$

$$\text{TNR} = \frac{45}{5+45} = 0.90$$

For multi-class classification

Confusion matrix

- Square matrix (k×k), where k is the # of classes. Each row corresponds to actual class, each column the predicted class.

example 2:

actual \ predicted	A	B	C	
C	40	5	5	50
B	10	35	5	50
A	5	10	30	45
	55	50	40	

diagonals → correctly classified

$$\text{Accuracy} = \frac{\text{correct predictions}}{\text{total \# of predictions}} = \frac{40+35+30}{\text{all}} = \frac{105}{145} \approx 0.7241$$

Precision:

For class i, Precision of class i is the measure of # of instances predicted as i which were actually in class i.

$$\text{Precision}_i = \frac{\text{predicted correctly as } i}{\text{predicted as } i} = \frac{TP_i}{\hat{i}}$$

For example 2,

$$\text{Precision}_A = \frac{40}{55} \approx 0.727 \qquad \text{Precision}_B = \frac{35}{50} \qquad \text{Precision}_C = \frac{30}{40}$$

Recall/sensitivity:

For class i: proportion of correctly predicted instances of i out of all actual instances of class i.

$$\text{Recall}_i = \frac{\text{predicted } i \text{ correctly}}{\text{actually } i} = \frac{TP_i}{i}$$

$$\text{Recall}_A = \frac{40}{50} = 0.80$$

f1-score:

For class i, HM of Precision_i and recall_i

How to average this shit?

1. Macro averaging

- Treats all the classes equally regardless of the # of instances in each class.

$$\text{macro-precision} = \frac{\sum_{i=1}^{K} \text{Precision}_i}{K} \qquad \text{macro-recall} = \frac{\sum_{i=1}^{K} \text{Recall}_i}{K} \qquad \text{macro-f1-score} = \frac{\sum \text{f1-score}_i}{K}$$

actual \ predicted	A	B	C	
C	40	5	5	50
B	10	35	5	50
A	5	10	30	45
	55	50	40	

	Precision	Recall	f1-score
A →	$40/55 = 0.7272$	$40/50 = 0.80$	0.7619
B →	$35/50 = 0.7$	$35/50 = 0.7$	0.7
C →	$30/40 = 0.75$	$30/45 = 0.66$	0.7058
macro →	0.7257	0.72	0.7226

NOTE: $\text{macro}_{f1} \neq HM(\text{macro}_{Pr.}, \text{macro}_{recall})$ ← not necessarily equal.

2. Micro averaging

- Aggregates the performance across all classes by summing the true positives, false positives, and false negatives before calculating the precision, the recall, or the f1-score.

$$\text{Total TP} = \sum TP_i$$

$$\text{micro-precision} = \frac{\text{Total TP}}{\text{Total TP} + \text{Total FP}}$$

$$\text{micro}_{f1} = HM(\text{micro}_{Pr.}, \text{micro}_{recall})$$

actual \ predicted	A	B	C	
C	40	5	5	50
B	10	35	5	50
A	5	10	30	45
	55	50	40	

$\text{Total TP} = 40 + 35 + 30 = 105$

$\text{Total FP} = (10+5) + (5+10) + (5+5) = 40$

$\text{Total FN} = (5+5) + (10+5) + (5+10) = 40$

$$\text{micro precision} = \frac{105}{105 + 40} = \frac{105}{145}$$ (40 = Total FP)

$$\text{micro recall} = \frac{\text{Total TP}}{\text{Total TP} + \text{Total FN}} = \frac{105}{145} = 0.7241$$

$$\text{micro}_{f1\ score} = \frac{105}{145}$$

NOTE:

Total FN = Total FP

$\text{micro precision} = \text{micro recall} = \text{micro f1}$

3. Weighted averaging

- Accounts for the # of instances in each class, giving more weight to classes with more instances.

← Balances size of class and individual performance ✓✓

weight_i = actual # of instances in i

$$\text{weighted precision} = \frac{\sum w_i \cdot \text{Precision}_i}{\sum w_i} \qquad \text{weighted recall} = \frac{\sum w_i \cdot \text{Recall}_i}{\sum w_i} \qquad \text{weighted f1 score} = \frac{\sum w_i \cdot \text{f1-score}_i}{\sum w_i}$$

$$\neq HM(\text{weighted precision}, \text{weighted recall})$$

actual \ predicted	A	B	C	
C	40	5	5	50
B	10	35	5	50
A	5	10	30	45
	55	50	40	

	Precision	Recall	f1-score
A →	$40/55 = 0.7272$	$40/50 = 0.80$	0.7619
B →	$35/50 = 0.7$	$35/50 = 0.7$	0.7
C →	$30/40 = 0.75$	$30/45 = 0.66$	0.7058

$$\text{weighted precision} = \frac{50(0.7272) + 50(0.7) + 45(0.75)}{145} = 0.7248$$

$$\text{weighted recall} = \frac{50(0.8) + 50(0.7) + 45(0.66)}{145} = 0.722$$

$$\text{weighted f1 score} = \frac{50(0.7619) + 50(0.7) + 45(0.7058)}{145} \approx 0.7231$$

ROC curve and AUC

(mostly used for binary classification)

Receiver Operating Characteristic curve

Area under the curve

$TPR = recall = sensitivity = \frac{TP}{TP+FN}$

$FPR = 1 - specificity = 1 - TNR = 1 - \frac{TN}{TN+FP} = \frac{FP}{TN+FP}$

ROC curve

- Graphical representation of a model's performance at different classification thresholds.
- Plots TPR vs FPR.
- A classification model typically outputs probabilities for each instance, and a threshold is applied to decide the final class of the instance.
- This threshold determines how strict the model is in classifying instances as +ve or -ve.

Threshold = 1.0 → predict all instances as -ve.

$TP = 0, FP = 0 \quad TPR = \frac{TP}{TP+FN} = 0 \quad FPR = \frac{FP}{FP+TN} = 0$

(0, 0) for threshold 1.

Threshold = 0.0 → all instances +ve.

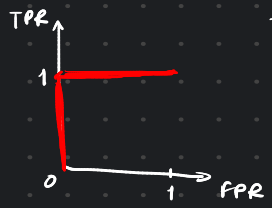
$TN = FN = 0$

$TPR = \frac{TP}{TP+FN} = 1 \quad FPR = \frac{FP}{FP+TN} = 1$

(1, 1) for threshold = 0.0

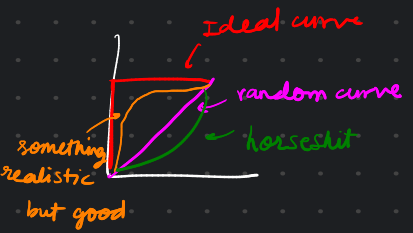
Ideal ROC curve

- The ideal ROC curve passes through the top-left corner of the plot, indicating high TPR and a low FPR. This means that the model performs well at all values of threshold.

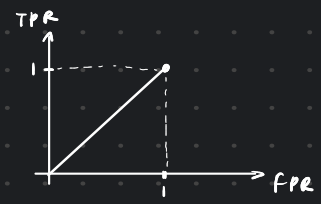


Threshold = 0 → TPR = FPR = 1

Threshold ↑ TPR ↓ FPR ↓



ROC curve for a random classifier



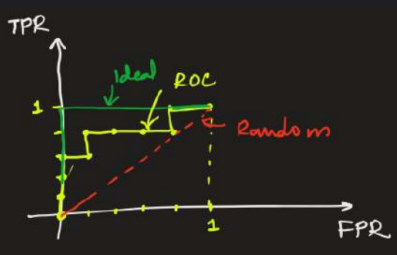
AUC

- AUC is a scalar value that summarizes the performance of the model based on the ROC curve. It ranges from 0 to 1.
- Higher the AUC, the better the model is in distinguishing between the positive and negative classes.

Eg:

Instance	Actual Label	Probability by the model
1	1	0.9
2	1	0.8
3	1	0.7
4	0	0.6
5	1	0.5
6	0	0.4
7	0	0.3
8	0	0.2
9	1	0.1
10	0	0.05

Threshold	TPR	FPR
1	0/5 = 0	0/5 = 0
0.95	0/5 = 0	0/5 = 0
0.85	1/5 = 0.2	0/5 = 0
0.75	2/5 = 0.4	0/5 = 0
0.65	3/5 = 0.6	0/5 = 0
0.55	3/5 = 0.6	1/5 = 0.2
0.45	4/5 = 0.8	1/5 = 0.2
0.35	4/5 = 0.8	2/5 = 0.4
0.25	4/5 = 0.8	3/5 = 0.6
0.15	4/5 = 0.8	4/5 = 0.8
0.05	5/5 = 1	4/5 = 0.8
0	5/5 = 1	5/5 = 1



$AUC = 0.2 \times 0.6 + 0.6 \times 0.8 + 0.2 \times 1$

$= 0.12 + 0.48 + 0.2$

$= \boxed{0.8}$

Log-loss (cross entropy loss)

Binary classification :

$$\text{cross-entropy loss} = -\frac{1}{N}\sum_{i=1}^{N}\left(y_i \log_e p_i + (1-y_i)\log_e(1-p_i)\right)$$

$y_i \rightarrow$ actual label
$p_i \rightarrow$ predicted prob.
$N \rightarrow$ # of observations.

e.g.

instance	actual label	predicted probability
1	1	0.8
2	0	0.3

1st instance $\rightarrow$ $1 \cdot \ln(0.8) + 0 \cdot \ln(0.2) = \ln(0.8)$

2nd instance $\rightarrow$ $\ln(0.7)$

$$\text{cross-entropy loss} = -\frac{1}{2}\left(\ln 0.8 + \ln 0.7\right) = 0.29$$

multi-class classificn :

$$\text{log loss} = -\frac{1}{N}\sum_{i=1}^{N}\sum_{j=1}^{K} y_{ij} \ln(p_{ij})$$

$K \rightarrow$ # of classes
$y_{ij} \rightarrow$ binary indicator (1 if the true label is j for i^{th} instance)
$p_{ij} \rightarrow$ predicted prob of jth class in i^{th} instance.

eg:

Instance	Actual Label	Predicted Probability A	B	C
1	A	0.7	0.2	0.1
2	B	0.1	0.6	0.3
3	C	0.2	0.3	0.5
4	A	0.8	0.1	0.1

Calculate the cross-entropy loss for this 3-class classification model.

$$\text{cross entropy loss} = -\frac{1}{4}\left(\log 0.7 + \log 0.6 + \log 0.5 + \log 0.8\right) = 0.45$$

Matthews correlation coefficient (MCC) ignore

$$MCC = \frac{(TP \times TN) - (FP \times FN)}{\sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)}}$$

MCC balances the amounts of TP, FP, FN, TN. It's particularly useful when the classes are imbalanced.

$-1 \le MCC \le 1$

- 1: Perfect prediction
- 0: Random prediction
- -1: Inverse prediction

Error metrics for multi-label classification

① Hamming Loss

It measures the fraction of labels that are incorrectly predicted (either missing from actual label or prediction label).

$$\text{Hamming loss} = \frac{1}{N \times k} \sum_{i=1}^{N} \sum_{j=1}^{K} 1 \cdot (y_{ij} \neq \hat{y}_{ij})$$

y_{ij} :- true label for j^{th} class in i^{th} instance

$\hat{y}_{ij}$:- predicted label for j^{th} class in i^{th} instance

$1 \cdot (y_{ij} \neq \hat{y}_{ij})$:- an indicator function that is 1 if the prediction is incorrect, otherwise 0

Eg:

Instances	Actual labels	Predicted labels
1	A, B	A, C
2	A, B, C	A, B
3	C	A, B, C

Sol. For Instance 1:-

class A :- $1(y_A \neq \hat{y}_A) = 0$

class B :- $1(y_B \neq \hat{y}_B) = 1$

class C :- $1(y_C \neq \hat{y}_C) = 1$

$$\sum_{j=1}^{K} 1(y_{ij} \neq \hat{y}_{ij}) = 2$$

For instance 2:- 1 instance 3:- 2

$$\text{Hamming loss} = \frac{2+1+2}{3 \times 3} = \frac{5}{9} \approx \boxed{0.56}$$

② Subset Accuracy (Exact Match Ratio)

is a stricter metric that measures the proportion of instances where all predicted labels exactly match the true labeles.

$$\text{Subset Accuracy} = \frac{1}{N} \sum_{i=1}^{N} 1 \cdot (y_i = \hat{y}_i)$$

$1(y_i = \hat{y}_i)$:- indicator function that is 1 if the predicted label set exactly matches the true label set, 0 otherwise

Eg:

Instance	True labels	Predicted labels	Exact Match?
1	A, B	A, B	✓
2	A, C	A	×
3	B, C	B, C	✓
4	A, B	A, B, C	×
5	C	B, C	×

Sol. Subset Accuracy $= \frac{2}{5} = \boxed{40\%}$ (0.4)

③ Coverage Error

measures how many labels, on average, must be included to cover all true labels. It indicates how far an average predicted ranking of labels is from covering the relevant labels.

$$\text{Coverage Error} = \frac{1}{N} \sum_{i=1}^{N} \left(\max_{j:\, y_{ij}=1} \text{rank}(j) \right)$$

rank (j) :- is the rank of the true label j in the predicted label ranking for instance i

Eg:

Instance	True labels	Predicted Ranking of Labels	max rank for all true labels
1	{A, C}	[A, B, C]	3
2	{B}	[B, C, A]	1
3	{A, B, C}	[C, A, B]	3

Coverage Error $= \frac{3+1+3}{3} = \frac{7}{3} \approx \boxed{2.33}$

④ Ranking Loss

evaluates how well the predicted label ranking aligns with the true labels. It measures the proportion of label pairs where the true label has a lower rank than a false label.

$$\text{Ranking Loss} = \frac{1}{N} \sum_{i=1}^{N} \left(\frac{1}{|Y_i| \times |\overline{Y_i}|} \sum_{\substack{(y_j, y_k) \in \\ Y_i \times \overline{Y_i}}} 1 \cdot (\hat{y}_j > \hat{y}_k) \right)$$

Y_i :- set of true labels for instance i

$\overline{Y_i}$:- set of false labels for instance i

$1(\hat{y}_j > \hat{y}_k)$:- indicator function that 1 if the predicted rank of true label j is greater than the rank of the false label k.

Eg:

Instance	True labels	Predicted Ranking of labels	Ratio of bad pairs
→ 1	{A, C}	[A, B, C]	1/2 = 0.5
2	{B}	[B, C, A]	0/2 = 0
3	{A, B, C}	[C, A, B]	0

Sol. Instance 1:- Y_1 = {A, C}, $\overline{Y_1}$ = {B}

$\{(A, B), (C, B)\}$ (A, B ✓; C, B ×)

Instance 2:- Y_2 = {B}, $\overline{Y_2}$ = {A, C}

$\{(B, A), (B, C)\}$ (B, A ✓; B, C ✓)

Instance 3:- Y_3 = {A, B, C}, $\overline{Y_3}$ = {}

$= \{\}$

$$\text{Ranking loss} = \frac{0.5+0+0}{3} \approx \boxed{0.167}$$

⑤ Precision, Recall, F1-score (Macro, Micro, Weighted Averaging)

$$\text{Precision}_i = \frac{\sum_{j=1}^{N} 1(y_{ij} = 1 \wedge \hat{y}_{ij} = 1)}{\sum_{j=1}^{N} 1(\hat{y}_{ij} = 1)}$$

← TP for i^{th} class

← Total True predictions for i^{th} class

$$\text{Recall}_i = \frac{\sum_{j=1}^{N} 1(y_{ij} = 1 \wedge \hat{y}_{ij} = 1)}{\sum_{j=1}^{N} 1(y_{ij} = 1)}$$

← TP for i^{th} class

← Total number of times i is a correct label in actual labels

$$\text{F1-score}_i = 2 \times \frac{\text{Precision}_i \times \text{Recall}_i}{\text{Precision}_i + \text{Recall}_i}$$

$$\text{Macro-Precision} = \frac{\sum_{i=1}^{K} \text{Precision}_i}{K}, \quad \text{Macro-Recall} = \frac{\sum_{i=1}^{K} \text{Recall}_i}{K}$$

$$\text{Macro-F1 score} = \frac{\sum_{i=1}^{K} \text{F1-score}_i}{K}$$

$$\text{Micro-Precision} = \frac{\sum_{i=1}^{N}\sum_{j=1}^{K} 1(y_{ij} = 1 \wedge \hat{y}_{ij} = 1)}{\sum_{i=1}^{N}\sum_{j=1}^{K} 1(\hat{y}_{ij} = 1)}$$

← overall TP

$$\text{Micro-Recall} = \frac{\sum_{i=1}^{N}\sum_{j=1}^{K} 1(y_{ij} = 1 \wedge \hat{y}_{ij} = 1)}{\sum_{i=1}^{N}\sum_{j=1}^{K} 1(y_{ij} = 1)}$$

$$\text{Micro-F1 score} = 2 \times \frac{\text{Micro Precision} \times \text{Micro Recall}}{\text{Micro Precision} + \text{Micro Recall}}$$

$$\text{Weighted Precision} = \frac{\sum_{i=1}^{K} w_i * \text{Precision}_i}{\sum_{i=1}^{K} w_i}$$

w_i : # instances where i^{th} label is present in actual labels

$$\text{Weighted Recall} = \frac{\sum_{i=1}^{K} w_i * \text{Recall}_i}{\sum_{i=1}^{K} w_i}$$

$$\text{Weighted F1-score} = \frac{\sum_{i=1}^{K} w_i * \text{F1-score}_i}{\sum_{i=1}^{K} w_i}$$